

The Theory of RAG

Teaching the AI New Tricks

BIG IDEA

A normal LLM is very smart, but it only knows what it was trained on and what you put into the current prompt. RAG solves this by searching your own documents first, then giving the most relevant information to the AI before it answers.

1. The Problem: The Isolated Professor

Before writing any code, you need to understand the limitation that RAG is designed to solve. A large language model can sound intelligent, but it is not automatically connected to your live company data, your private documents, or the latest schedules.

The Analogy: The Isolated Genius

Imagine hiring a brilliant Computer Science professor. This professor understands JavaScript, React, Node.js, APIs, databases, and programming logic. The professor can explain difficult ideas clearly and can write good code.

Now imagine you lock that professor inside a room. The room has no internet connection, no updated class schedule, no Evangadi course calendar, no pricing sheet, no student handbook, and no internal company notes.

The professor is still intelligent, but the professor is isolated. That means the professor can answer general programming questions, but cannot reliably answer questions about information they were never given.

Teaching point

An LLM can be excellent at reasoning and language, but still fail because it does not have the correct information available at answer time.

Limitation 1: Knowledge Cutoff

A standard LLM is trained on data available during its training period. After training, the model does not automatically learn new events, new schedules, new prices, or new policy changes unless that information is provided through another system.

- Student question: When does the next Evangadi batch start?
- Problem: The next batch schedule may have been created after the model was trained.
- Result: The model may say it does not know, or worse, it may guess a date that sounds believable.

This is called a knowledge cutoff problem. It is not because the model is broken. It is because the model is frozen with respect to the facts it learned during training.

Limitation 2: Private Data

Most useful company information is private. Course prices, internal schedules, student groups, company policies, payment rules, and support documents are usually not part of the public internet. If the data was not in the model training data, the model cannot know it by default.

- Student question: How much is the Full Stack class?
- Bad model behavior: It guesses a generic price based on general education costs.
- Correct behavior: It should only answer after checking the official Evangadi pricing document.

Limitation 3: Hallucination

A hallucination happens when an AI gives an answer that sounds confident but is not supported by the actual facts. The model is not intentionally lying. It is generating likely text based on patterns it has learned.

Hallucination is dangerous in company chatbots because users may trust the answer. A wrong price, wrong start date, or wrong policy can create confusion and support problems.

Question	Bad Answer	Why It Is Bad
When does Phase 3 start?	It starts next Monday.	The model guessed instead of checking the official schedule.
How much is the class?	Around \$5,000.	The answer uses general knowledge, not Evangadi data.
Can I join late?	Yes, always.	The real rule may depend on the current cohort policy.

The Old Solution: Fine-Tuning

Fine-tuning means training an existing model further on your own examples. In the professor analogy, it is like sending the professor to orientation and making them memorize company documents.

Fine-tuning can be useful, but it is not the best solution for information that changes often. If the schedule changes every month, retraining the model every month is slow, costly, and unnecessary.

- Fine-tuning is better for changing behavior, tone, style, or repeated output format.
- RAG is better for changing facts, private documents, schedules, prices, FAQs, and policies.

2. The Solution: What Is RAG?

RAG stands for Retrieval-Augmented Generation. It is a way to make an AI answer using external information that is retrieved at the moment the user asks a question.

Part	Meaning	Simple Explanation
Retrieval	Search for relevant information.	Find the right page, paragraph, or chunk from your documents.
Augmented	Add that information to the prompt.	Give the AI the evidence before asking it to answer.
Generation	Let the LLM write the final response.	The AI reads the context and produces a clear answer.

The New Solution: Give the Professor the Handbook

Instead of changing the professor's brain, we give the professor the correct page from the student handbook at the moment the question is asked.

1. The student asks: When does the Phase 3 React class start?
2. The system searches the Evangadi course calendar.
3. The system retrieves the paragraph that contains the Phase 3 start date.
4. The system sends both the student question and the retrieved paragraph to the LLM.
5. The LLM answers based on that paragraph.

Important

RAG does not require the model to permanently memorize your data. It gives the model the right data temporarily, inside the current request.

Why RAG Works

LLMs are very good at reading comprehension. If the answer is placed inside the prompt, the model can usually understand it, summarize it, and explain it in a helpful way.

RAG combines two strengths: a search system that finds relevant information, and a language model that explains that information naturally.

- The search system is responsible for finding evidence.
- The LLM is responsible for turning evidence into a useful answer.
- The prompt rules are responsible for stopping unsupported guessing.

3. The Core Concepts of RAG – (Deep Dive)

Before building the pipeline, you should understand three core ideas: the context window, context stuffing, and hallucination prevention.

Concept A: The Context Window

The context window is the maximum amount of text an LLM can read in one request. It includes the system prompt, user question, retrieved context, previous chat messages, and any instructions.

Even when a model has a large context window, sending too much information is still a bad habit. It can increase cost, slow down the response, and make it harder for the model to focus on the exact answer.

- Bad idea: Send a 10,000 - page textbook to the model for every question.
- Better idea: Search the textbook and send only the few paragraphs that are most relevant.
- RAG depends on selecting the right context, not simply sending all available text.

Concept B: Context Stuffing

Context stuffing means taking the retrieved text and placing it directly inside the prompt that goes to the LLM. This is the Augmented part of Retrieval-Augmented Generation.

```
System Prompt:
You are a helpful Evangadi assistant.
Use only the context below to answer the student.

Context:
[PASTE RETRIEVED TEXT HERE]

Student Question:
[PASTE USER QUESTION HERE]
```

This works because the LLM is not being asked to remember the answer from training. It is being asked to read the provided context and answer from that context.

Concept C: Hallucination Prevention

Hallucination prevention means adding strict rules that stop the AI from guessing when the answer is not in the retrieved context.

```
Strict Rule:
If the answer is not found in the provided context, say:
"I do not know based on the provided information."
Do not use outside knowledge.
Do not guess.
Do not invent details.
```

This rule is especially important for business chatbots. If the chatbot is only supposed to answer from Evangadi documents, it should not answer from general internet knowledge.

4. The RAG Architecture: The Pipeline

A RAG system can be understood as a library system for the AI. First, we organize the documents. Later, when a user asks a question, we search the organized library and give the most relevant information to the model.

Pipeline overview

Ingestion -> Chunking -> Embedding -> Storage -> Retrieval -> Generation

Step 1: Ingestion - Loading the Data

Ingestion means bringing information into the system. The data can come from text files, PDFs, Word documents, website pages, internal manuals, FAQs, or database records.

- Example file: evangadi-info.txt
- Goal: Read the text so the application can process it.
- Good practice: Save metadata such as file name, section title, date, and source type.

Metadata matters because it helps later when you want to show where the answer came from or filter results by document type.

Step 2: Chunking - Splitting the Text

Chunking means breaking a large document into smaller pieces. We do this because the AI cannot efficiently read every document for every question. Smaller chunks are easier to embed, store, search, and send to the LLM.

Bad Chunking	Good Chunking
Cuts a sentence in half.	Keeps complete ideas together.
Creates chunks with no context.	Uses overlap so meaning continues between chunks.
Makes chunks too large to search accurately.	Uses medium-sized pieces that are specific but meaningful.
Ignores paragraphs and headings.	Uses paragraphs, sentences, and headings as natural boundaries.

A common strategy is recursive chunking. The splitter tries to divide text by larger natural boundaries first, then smaller boundaries only when necessary.

1. Try to split by large sections or headings.
2. If the section is too large, split by paragraph.
3. If the paragraph is too large, split by sentence.
4. If the sentence is still too large, split by characters.

- Overlap means repeating a small part of the previous chunk in the next chunk.
- Example: If the last 50 characters of Chunk 1 are repeated in Chunk 2, the model has a smoother transition between ideas.

Why overlaps matter

If one chunk says "The Full Stack class price is" and the next chunk says "\$1000", the meaning can be lost unless the chunks overlap or are split better.

Step 3: Embedding - Translating Text into Meaning Vectors

After chunking, each chunk is converted into an embedding. An embedding is a list of numbers that represents the meaning of the text.

```
Text chunk:
"The Full Stack class price is $1000."

Embedding vector:
[0.13, -0.44, 0.82, 0.06, ...]
```

The same embedding model must be used for both the stored chunks and the user question. That way, the system can compare the meaning of the question with the meaning of each document chunk.

Step 4: Storage - Saving the Vectors

Storage means saving the original chunk text and its embedding vector so the application can search it later.

Column	Purpose
id	Unique identifier for the chunk.
content	The actual text of the chunk.
embedding	The vector representation, often stored as JSON in the learning version.
source	The file or document the chunk came from.
metadata	Optional details such as section, page, topic, or date.

For learning, we use MySQL with a JSON column. In production, a vector database is usually better for large-scale similarity search.

Step 5: Retrieval - Searching by Meaning

When the user asks a question, the system converts the question into a vector. Then it compares that question vector with every stored chunk vector and ranks the chunks by similarity.

The similarity score is often calculated using cosine similarity. A high score means the question and chunk are close in meaning.

Chunk	Similarity Score	Decision
Chunk A: Full Stack class price is \$1000.	0.92	Use this chunk. Very relevant.
Chunk B: React phase starts on Saturday.	0.63	Maybe useful, but not about price.
Chunk C: Office holiday schedule.	0.18	Ignore. Not relevant.

- Top-k means selecting the top few chunks, such as the top 3 most similar chunks.
- Threshold means ignoring chunks below a minimum score, such as 0.70.
- A threshold helps prevent weak or irrelevant context from confusing the LLM.

Step 6: Generation - Producing the Final Answer

Generation is the final step. The system puts the retrieved chunks into a prompt, adds the user question, and asks the LLM to answer using only the provided context.

```
You are a helpful Evangadi Professor.  
Use ONLY the context below.  
If the answer is not in the context, say "I do not know based on the provided information."  
  
Context:  
[Top 3 retrieved chunks]  
  
Question:  
[Student question]  
  
Answer:
```

This is the moment where search and language generation work together. The search system finds evidence. The LLM explains the evidence.

5. Detailed End-to-End Example

Let us walk through a complete example using an Evangadi chatbot.

User Question

```
Student: How much is the Full Stack class?
```

What Happens Behind the Scenes

1. The application receives the student question.
2. The question is converted into an embedding vector.
3. The application compares that vector with stored document vectors.
4. The top matching chunks are selected.
5. The selected chunks are inserted into the prompt as context.
6. The LLM answers using only that context.

Retrieved Context Example

Chunk 1:

The Full Stack Development class costs \$1000. Students may contact support for payment options.

Chunk 2:

The next Full Stack batch begins on Saturday. Orientation details will be sent by email.

Chunk 3:

Students should complete registration before the start of the batch.

Final Answer Example

Answer:

The Full Stack Development class costs \$1000. For payment options, students should contact support.

Notice that the answer is not a random guess. It is grounded in the retrieved context.

If the Answer Is Not Found

Student: Who is the CEO of Apple?

Retrieved Context:

Only Evangadi course information is found. No Apple CEO information appears.

Correct Answer:

I do not know based on the provided information.

This behavior may feel strict, but it is exactly what we want in a controlled company chatbot. The chatbot should stay inside the knowledge base it was designed to use.

One-Minute Explanation of RAG – Final takeaway

RAG is like giving an AI an open-book exam. The AI may not know a company schedule or price from memory, so the application first searches the company documents. It finds the most relevant paragraphs, places them into the prompt, and tells the AI to answer only from those paragraphs. This makes the answer more accurate, easier to update, and safer than guessing.

RAG is the bridge between a general AI model and your own reliable knowledge base. It does not make the model magically know everything. It gives the model the right information at the right time.